I Kept Hindsight Suggestions Separate From Confirmed Fixes

Designing an incident-memory system where a suggested fix is evidence for an investigation, never proof that an incident was resolved.

Engineering Notes · Design & Architecture

FastAPI · React · Agno · Hindsight memory layer

The easiest way to poison an incident memory system is to write every plausible answer back into it. I built this one around a stricter rule: a suggested fix is evidence for an investigation, not evidence that the incident was resolved.

That sounds obvious until the first useful answer arrives. A model sees `db_pool=exhausted`, recalls a similar outage, and proposes retry jitter and a larger connection pool. The proposal may be excellent — or exactly wrong for this deploy. Retain it as verified, and the next engineer inherits a confident echo of an untested guess.

The design decision that shaped this system was to make those states impossible to confuse: analysis does not write memory; a confirmed resolution is a durable incident record; and feedback is its own kind of evidence. The Hindsight GitHub repository (https://github.com/vectorize-io/hindsight) is the memory layer that makes those records useful later, but it does not get to decide what counts as operational truth. We do.

The system has one job: turn precedent into a reviewable suggestion

The application is an incident-resolution service: a React console over a FastAPI backend. An engineer pastes a title, impact, severity, and a log excerpt. The backend formats that into a query, asks Hindsight for related operational history, then hands the current incident and the recalled records to an Agno-managed model. The reply is two or three proposed resolutions, each with a confidence value and references to the records behind it.

Hindsight handles server-side embedding and fact extraction on retain and recall, so the service runs no separate embedding pipeline. The larger benefit is architectural: the memory bank is a durable record of incidents and decisions, not an opaque transcript. The Hindsight documentation (https://hindsight.vectorize.io/) describes the retrieval and retention primitives; the Vectorize agent memory overview (https://vectorize.io/what-is-agent-memory) explains why a system needs more than a model's context window.

The API deliberately starts with a non-writing operation:

```
@app.post("/api/incidents/new", response_model=AnalyzeResponse)
def create_incident(incident: IncidentNew, top_k: int = Query(default=3,
ge=1, le=10)):
    try:
        return analyze_incident(incident, top_k=top_k)
    except RuntimeError as exc:
        raise HTTPException(status_code=500, detail=str(exc))
```

`POST /api/incidents/new` returns a short-lived incident ID, suggestions, and the recalled records. It does not persist the incident merely because somebody asked a question. That boundary prevents the system from learning from every half-formed report and every generated answer.

The other two writes are explicit. `POST /api/incidents/resolve` receives a real resolution after the incident is fixed. `POST /api/incidents/feedback` receives a helpful or not-helpful verdict for a particular suggestion. The frontend keeps the original incident payload with the response so it can resend it with the verdict; the server needs no hidden session.

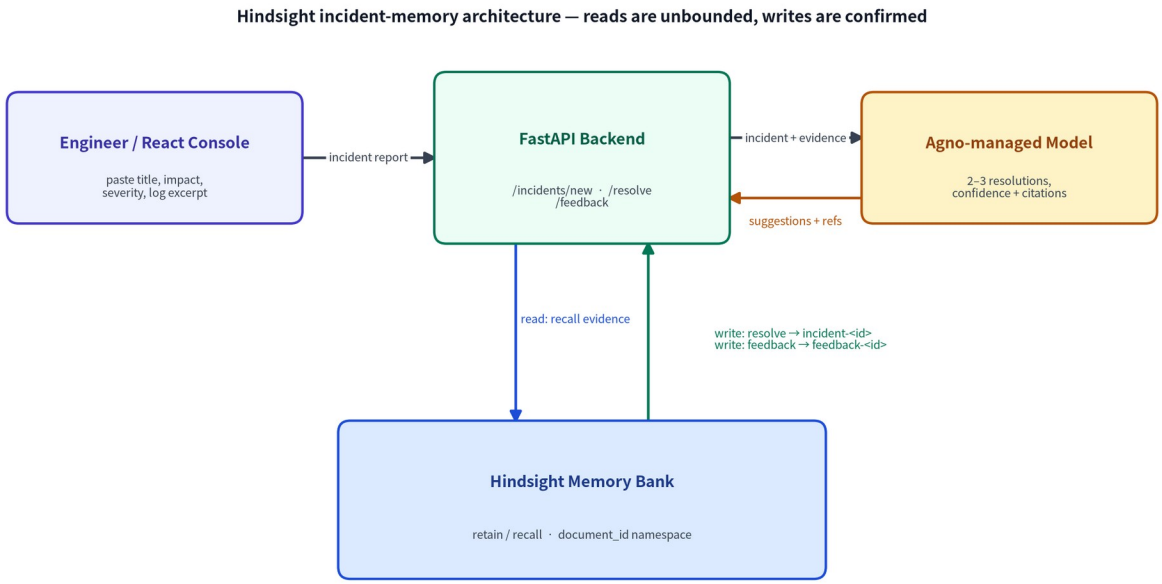


Figure 1 — The service recalls evidence from Hindsight on every analysis, but only confirmed resolutions and verdicts are written back.

I made confirmation a write boundary

The only route that creates a confirmed-fix record calls `persist_resolution`. It stores enough context to judge whether the precedent applies: title, severity, description, log excerpt, and the resolution.

```
def persist_resolution(client, bank_id, incident_id, incident, resolution):
    content = format_incident_for_memory(incident_id, incident, resolution)
    client.retain(
```

```
        bank_id=bank_id,
        content=content,
        document_id=f"incident-{incident_id}",
        metadata={"status": "resolved", "severity": incident.severity.value},
    )
    return f"incident-{incident_id}"
```

The document ID is not cosmetic. `incident-{incident_id}` makes a second submission of the same confirmation an upsert instead of a duplicate memory. During incident response, retries are normal; idempotency beats an after-the-fact deduplication heuristic.

I also chose a document-oriented archive. Semantic recall surfaces fact fragments, but the incident explorer fetches the original documents — when a suggestion cites a past outage, the engineer reads the full context, not a compacted fact. In operational tooling, the full record is the audit trail.

Hindsight earns its place here: a recall interface over retained operational records, without forcing a local vector database, a chunking worker, and another store for extracted facts into the application. The system still owns the schema of trust: `status=resolved` means something different from everything else in the bank.

Feedback is evidence, not a resolution

Feedback has a different failure mode. It is tempting to record a thumbs-up as proof that a suggestion worked. A person may have found an answer useful because it narrowed the search, described a symptom accurately, or led elsewhere. A thumbs-down is even less like a resolution, but still valuable: this framing or candidate fix was not useful here.

So feedback is retained under a different document namespace and metadata shape:

```
content = (
    f"User feedback on incident {incident_id}: {verdict} | "
    f"Rated suggestion index: {suggestion_index} | "
    f"{format_incident_for_query(incident)}"
)
client.retain(
    bank_id=bank_id,
    content=content,
    document_id=f"feedback-{incident_id}",
    metadata={"status": "feedback", "helpful": str(helpful).lower()},
)
```

Notice what is absent: we never promote the generated prose into a confirmed runbook step. We retain the incident context, the verdict, and which ranked suggestion was assessed — a durable positive or counterexample that preserves the difference between “this was helpful” and “this fixed production.”

That distinction also keeps the incident lifecycle legible: a new incident triggers recall and proposals but writes nothing; feedback retains a verdict about a suggestion; resolution retains the confirmed fix.

I prefer this over a single `saveMemory()` path with an ever-growing pile of optional fields. Separate operations make the caller state its intent, make authorization easier to reason about, and give us a place to add retention or retrieval policy by record type — without inferring truth from free-form text later.

Retrieval needs an evidence budget, not a magic number

One implementation detail was surprisingly easy to get wrong. Hindsight recall does not expose a native `top_k` parameter in this integration. It uses a recall budget and a token ceiling to decide how much context to return; the service then limits the records it passes onward to the model and UI.

```
resp = client.recall(
    bank_id=bank_id,
    query=query_text,
    budget=budget,
    max_tokens=max_tokens,
    include_chunks=True,
)
results = getattr(resp, "results", []) or []
for record in results[:max(top_k, 0)]:
    similar.append(SimilarIncident(
        memory_id=getattr(record, "id", "") or "",
        text=getattr(record, "text", "") or "",
        document_id=getattr(record, "document_id", None),
    ))
```

It is not just a parameter-name quirk: recall breadth and prompt cardinality are different controls. The first sets how much evidence Hindsight returns; the second constrains what reaches the generator and the engineer. Conflating them makes latency and answer quality harder to tune.

The prompt makes the evidentiary requirement explicit: every proposed resolution must carry references to the recalled records. If nothing is recalled, the instructions require low confidence rather than pretending generic advice came from local history. The model may synthesize an answer; it may not erase the difference between precedent and conjecture.

What an interaction looks like

The deployed console is live at incident-agent-o8vf.vercel.app (https://incident-agent-o8vf.vercel.app/).

An engineer reports: “Checkout requests return 504s after a deploy; `payments-api` has 200 active database connections and retries are synchronized.” The service turns the title, description, severity, and logs into one Hindsight query. A previously retained record about connection-pool saturation and missing retry jitter is a plausible match. Without memory, that diagnosis starts from scratch; with it, the precedent and its citation land in the same view.

The response may contain a candidate such as: enable jittered backoff, inspect the pool limit, and scale only after confirming connection saturation. Crucially, it also names the recalled record that supports the advice — the console shows that relationship, not just a confidence number.

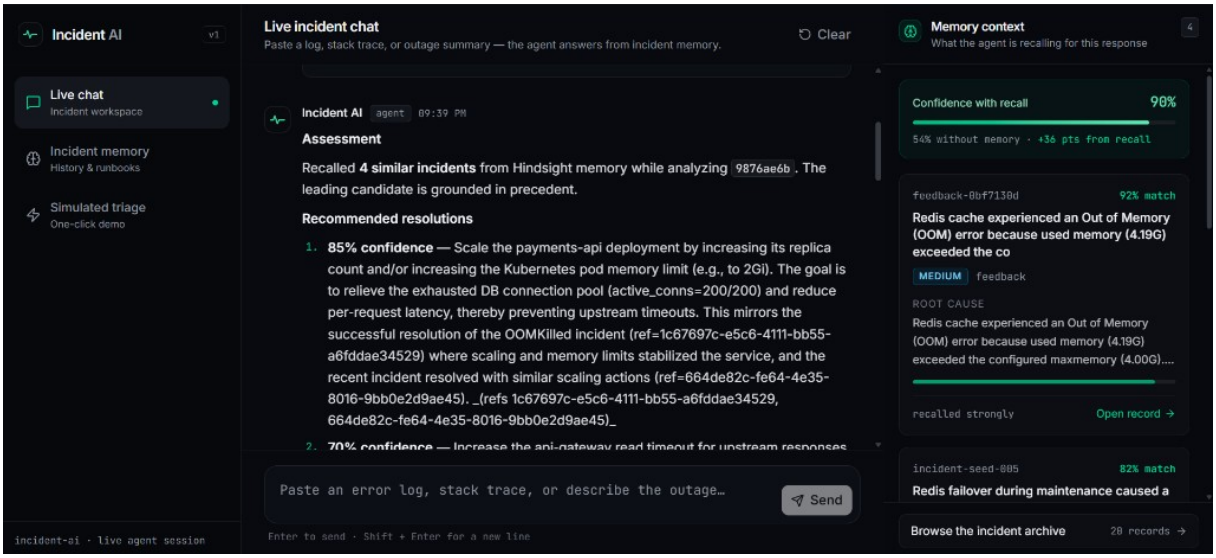


Figure 2 — The live console: an assessment grounded in four incidents recalled from Hindsight, with 85% and 70% confidence and explicit record references.

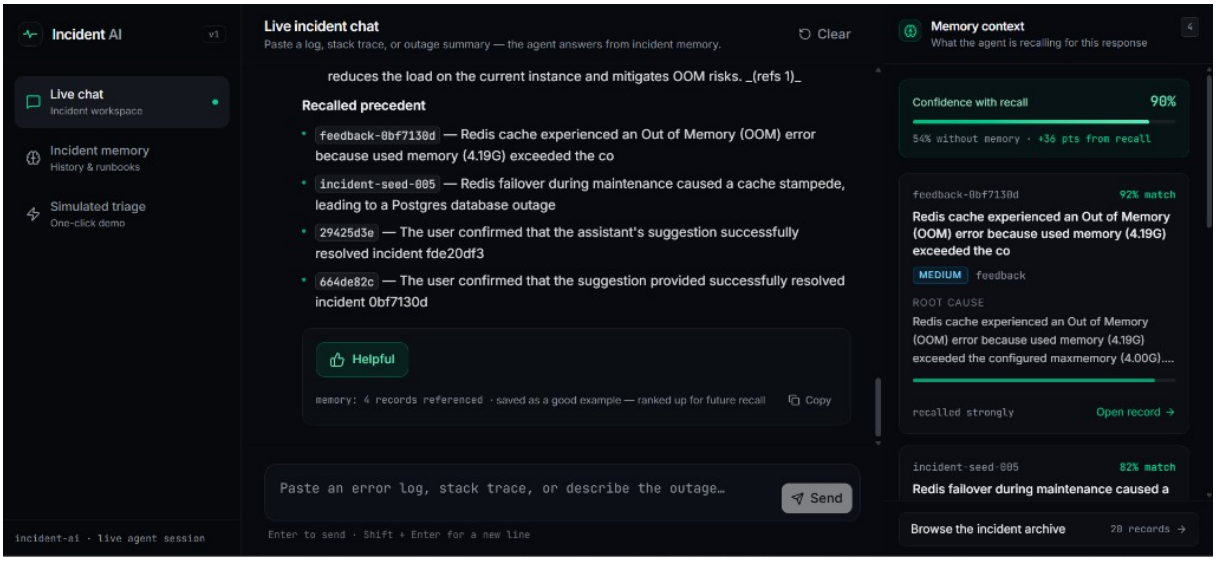


Figure 3 — Recall made visible: 90% confidence with memory versus 54% without, and the top precedent matching at 92%.

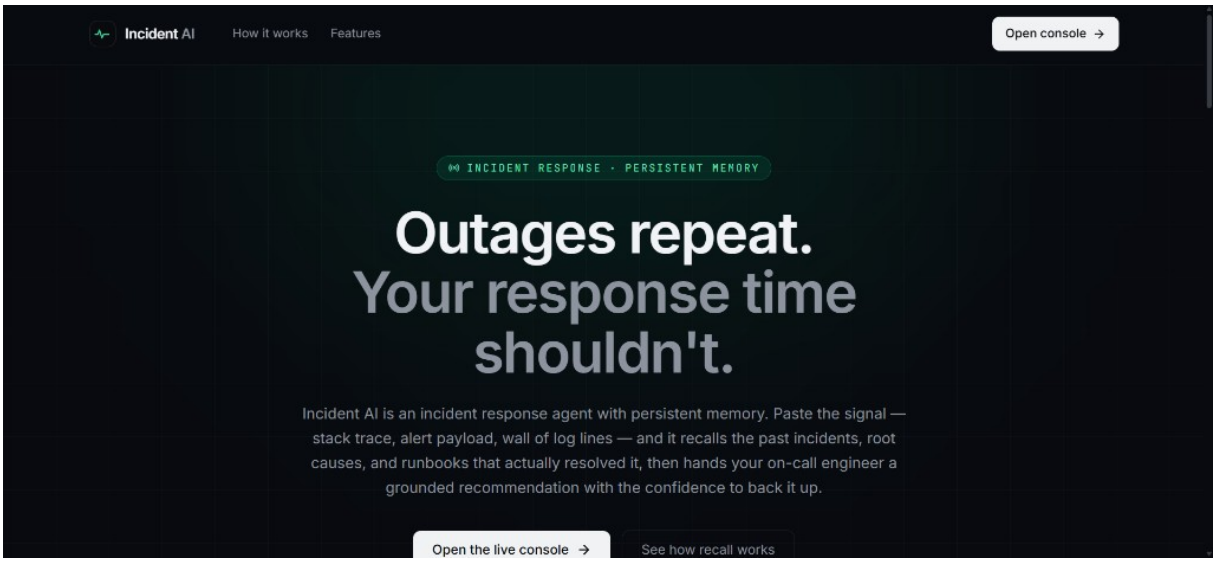


Figure 4 — The deployed product framing: "Outages repeat. Your response time shouldn't."

If the suggestion was useful, the engineer records that verdict against suggestion zero. If the remediation is validated, someone calls `/resolve` with the confirmed outcome. Two inspectable records result: `feedback-<id>` and `incident-<id>`. If the idea was wrong, the negative verdict remains a counterexample — it never masquerades as a fix.

I am not claiming this guarantees faster resolution. Those are outcomes to measure in production, not properties conferred by a retrieval call. What the system guarantees by construction is provenance: a generated proposal, a user verdict, and a confirmed resolution occupy different states.

What I learned building it

- **A useful answer is not a verified answer.** Retention needs a trust model before it needs clever retrieval. If the write path does not distinguish proposal from confirmation, bad guidance will eventually acquire the appearance of history.
- **Make the first analysis non-persistent.** Engineers investigate noisy, incomplete reports. A write on every query turns that noise into durable context and makes later recall less trustworthy.
- **Store the original incident with the verdict.** The feedback endpoint is stateless: the caller resends the title, description, logs, severity, incident ID, and selected suggestion.
- **Treat citations as part of the response contract.** A reference is not decoration under an answer. It is the path an on-call engineer uses to decide whether the precedent actually applies.
- **Tune recall and prompting separately.** Hindsight's recall budget sets evidence breadth; the application-level slice sets how much of that evidence reaches the model and operator. Configure and observe them as separate levers.

I did not build this to turn incident response into a conversation with a model. I built it to make prior operational experience easier to retrieve without laundering guesses into institutional memory. Hindsight supplies the durable recall layer; explicit write boundaries keep that memory worth trusting.